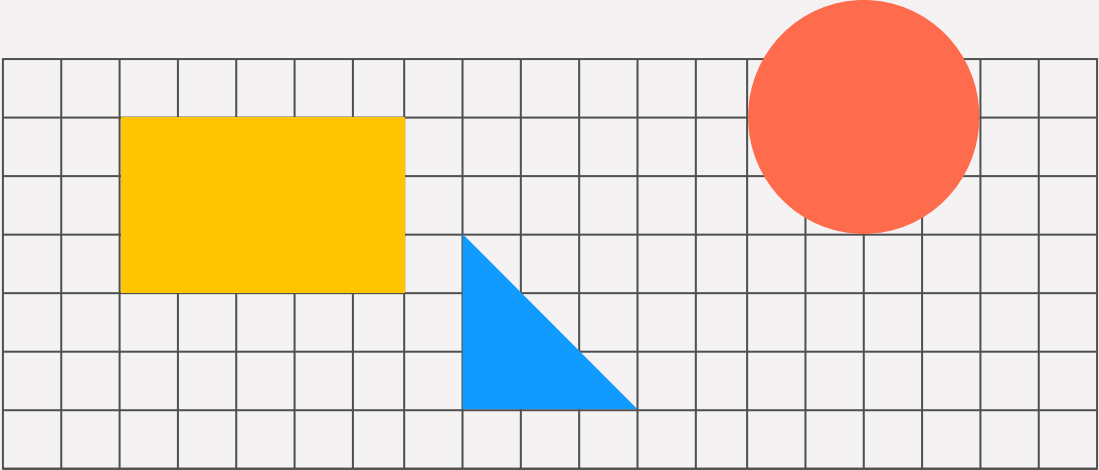


CSCI 580

Artificial Intelligence

Final Project Presentation:
MNIST Handwritten Digit Image
Detection via MLP

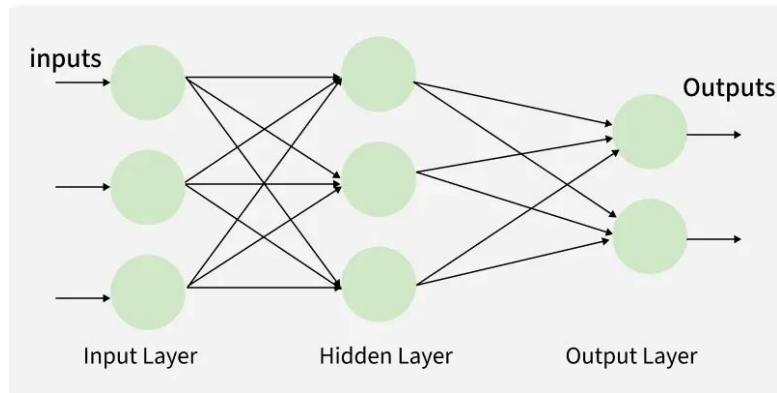
Joseph Lorenzo Bautista
Jesus Javier G. Ramirez
Kevin Xiong
Adam T. Lepage



Architecture of MLP

What is an MLP?

- Fully connected feedforward neural network.
- Every neuron in one layer connects to every neuron in the next.
- It is comprised and made of:
 - Hidden Layers
 - Output Layers
- Each connection has a learnable weight where each neuron has a learnable bias.
- Non linear activation function (ReLU) between layers makes so that its able to learn complex patterns.

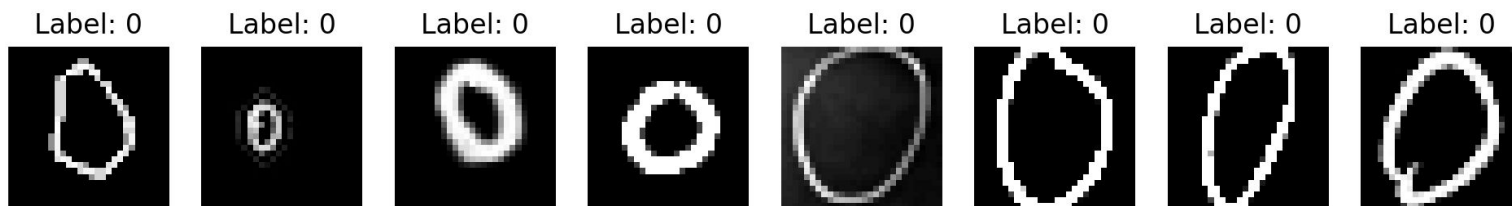


Architecture of MLP

Our MLP's Architecture

- Inputs Layers of 784 Neurons (28 x 28 pixel image flattened)
- ↓
- 2 Hidden Layers with ReLU Activation
- ↓
- Dropout of 20% in order to prevent overfitting.
- ↓
- Output Layer of 10 neurons (one score per digit from 0 to 9)

Same trained network used on BOTH MNIST data set and the group's handwritten images.



Architecture of MLP

How our MLP Learns

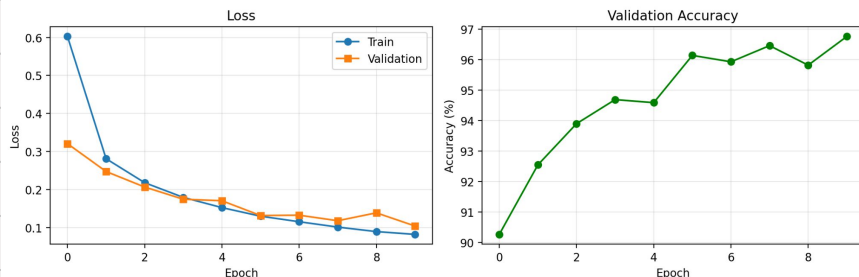
- **Forward Pass:** input flows and moves through layers → makes 10 scores (one per digit).
- **Loss Function:** CrossEntropyLoss measures how wrong the predictions are.
- **Backpropagation:** calculates the gradients and see how each weight contributed to the error.
- **Optimizer:** adjusts the weight in order to reduce the error.

This cycle iterates and repeats over many epochs.

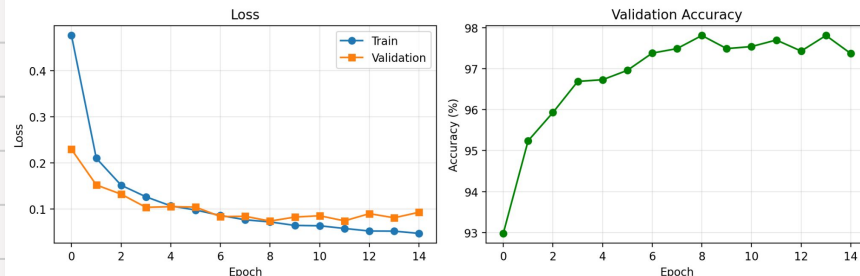


Training + Validation & Loss Evaluation

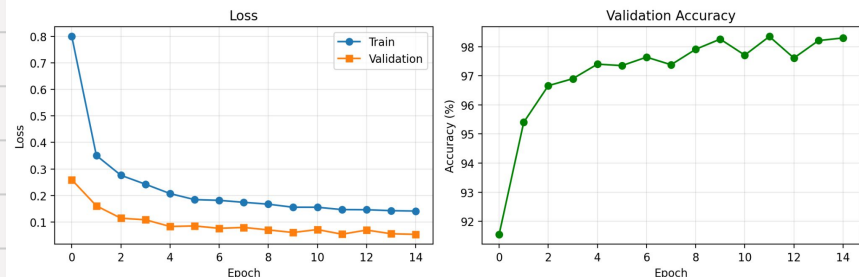
Iteration #1 - Baseline MLP



Iteration 2 - Larger Network + Dropout



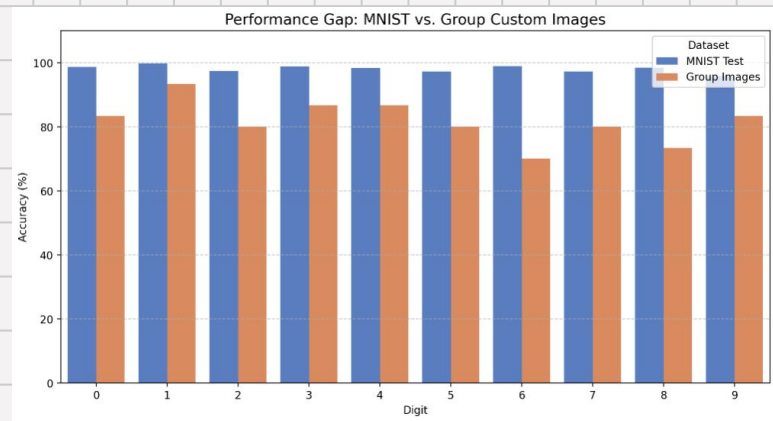
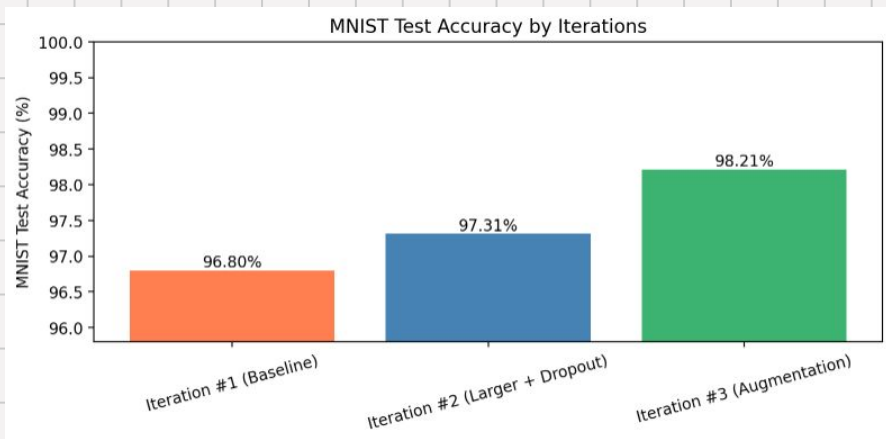
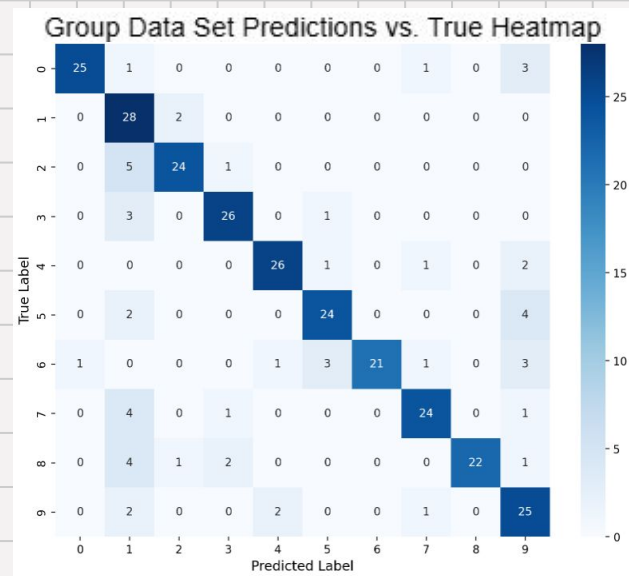
Iteration #3 - With Data Augmentation



- 50,000 images, 15 epochs, predict → measure error → adjust weights → repeat
- Iteration 3 randomly rotates and shifts images so the model learns instead of just remembering the shapes/pixels.
- Loss decreased steadily in all 3 iterations meaning that the model is learning.
- Validation tracks training closely so that it leads to no overfitting.
- Validation accuracy improves across iterations from 96.8% to 97.31% to 98.21%.

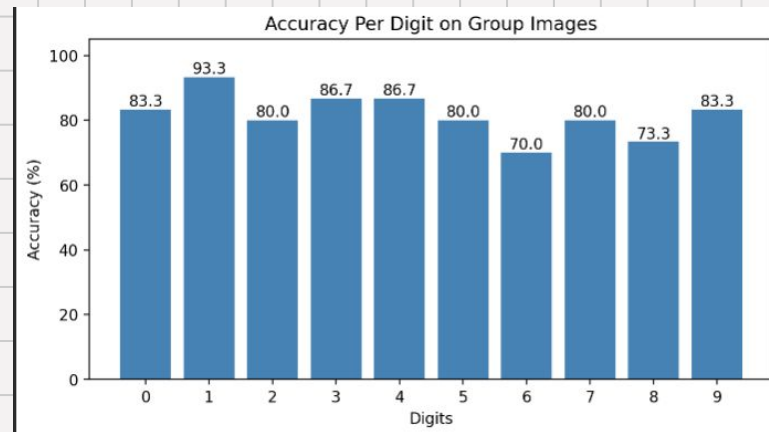
Testing Results

- The best-performing model (Iteration #3) was tested on the handwritten dataset of 300 images
- Accuracy for hand written images is 81.67% (245 correct out of 300)
- There is a 16.48% drop in accuracy when moving from the MNIST set to the handwritten images.

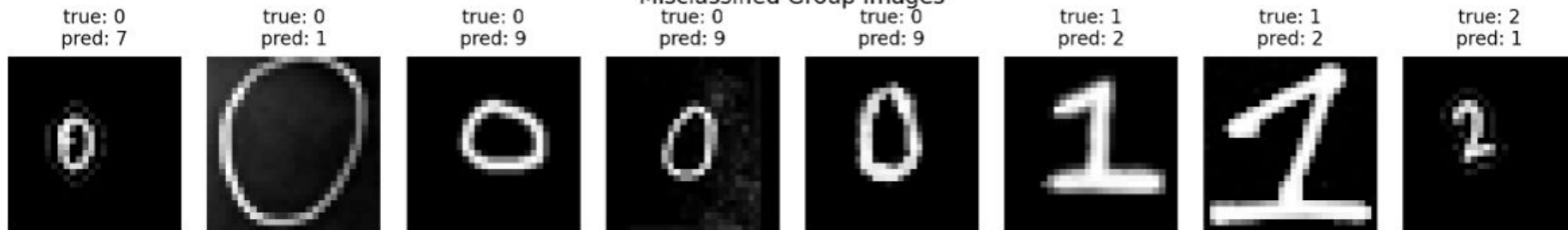


Analysis

- Performance varies by digit
- Errors caused by similar shapes and messy handwriting
- Conclusion: High test accuracy does not always translate to real-world performance due to unseen data variability



Misclassified Group Images



Critical Component

ProjectDataLoader

- Reads the PNGs and JPGs files from folder
- Parses label from the filenames first character
- Converts grayscale in order to match the MNIST
- Resizes and make sure that the image are 28x28
- Inverts the color so it makes sure that the photos are black and white

```
def ProjectDataLoader(folder_path="digits_data", invert=True, verbose=True):
    images = []
    labels = []

    # Grabs the sorted list of PNG files
    all_files = os.listdir(folder_path)
    image_files = sorted(f for f in all_files if f.lower().endswith((".png", ".jpg", ".jpeg")))

    for fname in image_files:
        try:
            label = int(fname[0])
        except ValueError:
            print(f'Skipping {fname}: Could NOT Parse Label From Filename')
            continue

        path = os.path.join(folder_path, fname)
        img = Image.open(path).convert("L")
        img = img.resize((28, 28))
        arr = np.array(img, dtype=np.uint8)

        if invert:
            arr = 255 - arr

        images.append(arr)
        labels.append(label)

    images = np.array(images)
    labels = np.array(labels, dtype=np.int64)

    if verbose:
        print(f"Loaded {len(images)} Images From '{folder_path}'")
        print(f"    images.shape = {images.shape}")
        print(f"    labels.shape = {labels.shape}")

    return images, labels
```